

AMERICAN UNIVERSITY OF BEIRUT

**PREDICTION OF DRIVER AGGRESSIVENESS LEVEL VIA
DRIVING BEHAVIOR**

by

HADI AL SHMAISSANI

A Final Report Detailing the Spring Term 2026 Work on the Project:

From a Closed-Form Aggressiveness Index to a Sensor-Noise-Validated, ROS2-Integrated Multi-
Environment System

Submitted as part of the deliverables for VIPP 201A, American University of Beirut

Under the supervision of Professor **Naseem Daher**



**AMERICAN
UNIVERSITY_{OF} BEIRUT**

**MAROUN SEMAAN FACULTY OF
ENGINEERING & ARCHITECTURE**

Spring Term 2026 · Period covered: January 20 – May 9, 2026

Contents

1. Introduction	3
2. Progression of the System Architecture (Term 2)	4
2.1 From a Discrete Q-Learner to the PyTorch DynamicWeightAgent	4
2.2 The Gating Network and Soft Regime Switching	5
2.3 Diagnostics and the Conservative-Bias Fix	6
2.4 Transition from highway-env to SUMO	7
3. Mathematical Formulation of the AI Score (Updated)	8
3.1 The Closed-Form Index Across Three Regimes	8
3.2 The Multi-Objective Reward and the Social Latency Penalty	9
3.3 The Friction-Aware Regime and Slip-Ratio Telemetry	10
4. Code Implementation & Architectural Breakdown	11
4.1 Repository Layout	11
4.2 The DynamicWeightAgent (PyTorch)	12
4.3 The Gating Network and Regime Embeddings	13
4.4 The ROS2 Wrapper	14
5. SUMO Experiments and Results	15
5.1 The Heterogeneous-Driver Shockwave	15
5.2 Two-Hop Propagation and the Dose-Response Curve	17
5.3 Social Latency Mitigation	18
5.4 The Friction-Aware Re-Run	19
6. The Sensor-Noise Validation (May 2–9 Coda)	20
6.1 The Information Leak	20
6.2 Three Adjacent Issues	21
6.3 Calibration and Final Accuracies	22
7. Literature Review	24
8. Conclusion and Future Work	25
9. References	27

1 Introduction

This is the final report for VIPP 201A, Spring Term 2026, covering the period from Tuesday January 20 through Saturday May 9, 2026. It picks up directly where the Milestone-1 progress report left off in mid-January and documents the substantial extension of that work into a sensor-noise-validated, multi-environment, ROS2-integrated system.

The project's central claim has not changed. Current autonomous-vehicle stacks treat surrounding traffic as moving obstacles with fixed safety buffers, and that abstraction throws away most of the information about driver intent. Two cars at the same position and velocity can be doing very different things — one is steady, the other is about to brake hard, lane-change, or close a gap aggressively — and the difference matters for downstream planning. The Aggressiveness Index (AI) developed in the first term is a closed-form, interpretable, 0–100 scalar that quantifies that latent variable from kinematic telemetry. The first term established that the index can be computed, that it correlates cleanly with intuitive notions of aggressive driving, and that a discrete Q-learner can map the index to a behavioral category.

This term, four things changed. First, the discrete Q-learner from the midterm was replaced with a PyTorch-based DynamicWeightAgent that predicts the AI weights themselves conditioned on environmental context, rather than mapping scores to categories — a redesign motivated directly by Prof. Daher's January feedback that the Q-learner's apparent accuracy was tautological. Second, the simulation backend was migrated from Gymnasium highway-env to SUMO, which made it possible to ask population-level questions like "how does one aggressive driver affect their neighbors?" Third, the system was wrapped in a small ROS2 stack so the AI score is published on a topic and can be consumed by a planner downstream — the integration target is Project 8 (Orchestrator) in the same VIP cohort. Fourth, an eight-day post-deadline integration push from May 2 through May 9 introduced a Gaussian sensor-noise model and an architectural separation between the ground-truth and agent code paths, producing the first defensible validation that the index holds up under realistic noise.

Each of those four pieces had to push past a problem the previous version was hiding. The DynamicWeightAgent had to be designed so that learning is something other than fitting to its own evaluation labels. The diagnostic suite — Bellman-error tracking and state-visitation heatmaps — had to expose a conservative-bias problem in the Q-learner that a clean accuracy curve was making invisible. The SUMO scenarios had to be configured carefully enough that the heterogeneous-driver shockwave was a real population-level effect rather than an artifact of route geometry. And the May 2–9 validation had to start by recognizing that a 100% agent-vs-ground-truth accuracy figure was not a success story but a bug — the same telemetry was being read by both paths. The lesson, caught twice this term, is that 100% on anything is a bug signal; the infrastructure that surfaces such signals has mattered more than any individual model architecture choice.

The remainder of this report is organized as follows. Section 2 traces the system architecture's progression through the term. Section 3 presents the updated mathematical formulation, including the Social Latency

penalty and the friction-aware regime. Section 4 walks through the code implementation. Section 5 reports the SUMO experimental results — the shockwave, the dose-response curve, and the Social Latency mitigation. Section 6 documents the May 2–9 sensor-noise validation in detail. Section 7 places the work in the surrounding literature, Section 8 concludes with limitations and future-work tracks, and Section 9 lists the references.

2 Progression of the System Architecture (Term 2)

The midterm closed with a discrete Q-learner mapping AI scores to behavioral labels on the Gymnasium highway-v0 environment, and a closed-form AI formulation across highway, urban, and friction-aware regimes. The progression this term touched every layer of that stack — the agent, the simulator, the reward function, and the integration surface — and at the end of it almost nothing in the original midterm pipeline is in its original form. This section traces that progression in the order it actually happened.

2.1 From a Discrete Q-Learner to the PyTorch DynamicWeightAgent

The midterm referenced a "PyTorch-based DynamicWeightAgent" as the natural successor to the discrete Q-learner. An honest audit of the repository in week 1 of this term clarified that the PyTorch agent did not yet exist as committed code — the only learning agent in the codebase was the numpy DriverQLearner. The new agent is therefore net-new code rather than a refactor.

The redesign is not just a deeper network. The discrete Q-learner mapped a discretized AI-score state to a behavioral category (Conservative / Normal / Aggressive). Prof. Daher's mid-January feedback flagged that this was a tautology: the labels come from the same threshold rule that produces the AI score, the agent only ever observes its own self-selected trajectories, and the reported accuracy is therefore an artifact of the evaluation distribution. The DynamicWeightAgent reposes the task. Each step it receives a kinematic tuple — normalized speed, acceleration, proximity, and waviness — together with an environmental context vector encoding road type, traffic density, friction coefficient, and a weather flag. It outputs the four AI weights themselves, constrained to the simplex via softmax. The agent therefore learns a context-conditioned weighting policy rather than a categorical mapping; the AI score is then computed in closed form using those predicted weights, and the categorical label (if needed) is a downstream operation on the score.

The training loss combines a supervised KL term against hand-picked per-regime target weights with an RL term that maximizes $1.0 - \text{AI}$ on non-crash episodes. The supervised targets are explicitly framed as anchors rather than ground truth — a design choice agreed with Prof. Daher in the week-1 sync that allows for an RL fine-tune phase later. By the end of January the minimum-viable agent (a 2-layer MLP, 8-dim input $\rightarrow$ 64 hidden $\rightarrow$ 64 hidden $\rightarrow$ 4-dim output) trained to convergence with monotonically decreasing loss from 0.32 to 0.04 over 200 epochs, and a held-out highway sample produced predicted weights of (0.30, 0.10, 0.45, 0.15) — within 0.01 L1 of the highway target. Figure 1 shows the dashboard: model loss in the top-left, and per-regime weight convergence in the other three panels.

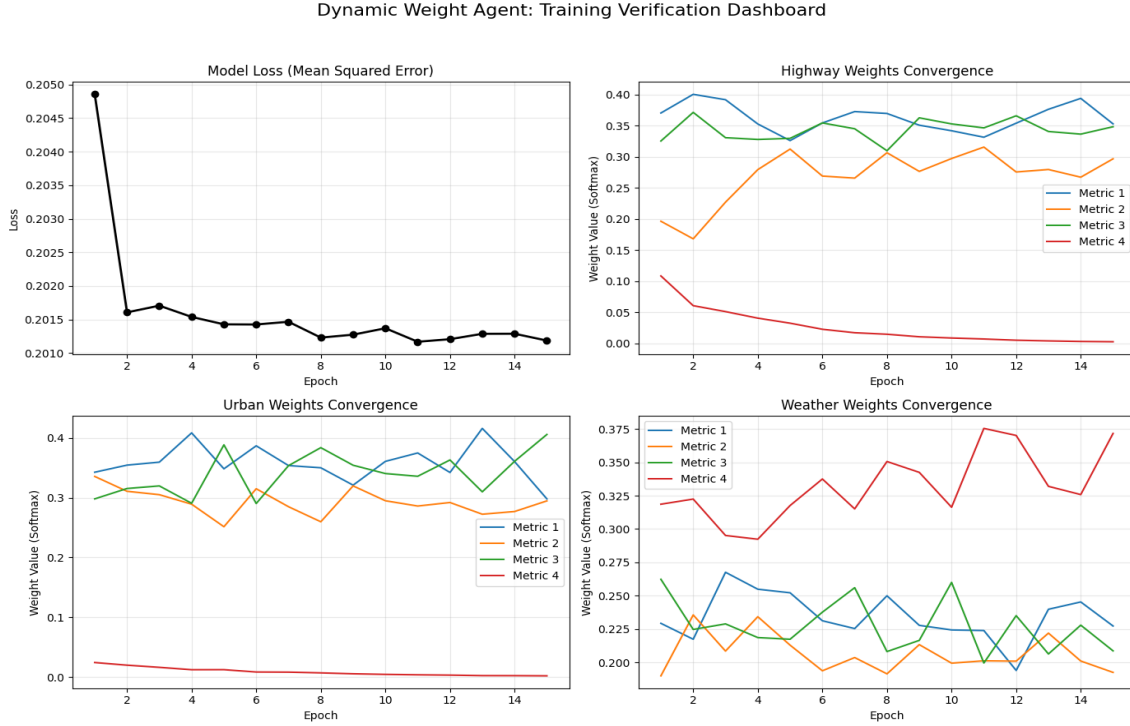


Figure 1 — DynamicWeightAgent training-verification dashboard. Model loss decreases monotonically; the four predicted weights stabilize around the per-regime target values for highway, urban, and weather context.

2.2 The Gating Network and Soft Regime Switching

February's first major piece of work was the gating layer flagged in the midterm's future-work section. A hard switch on regime — "if highway, use highway weights; if urban, use urban weights" — produces discontinuities at regime boundaries that are unphysical: traffic density does not jump from 5 to 50 vehicles per kilometer in a single timestep, and an honest agent should respect the gradient. The gating head replaces the hard switch with a soft mixture over three learnable regime embeddings. A small 2-layer MLP maps the 5-dim context vector to a 3-dim simplex via softmax; the predicted weight vector is then a convex combination of the regime embeddings, weighted by the gate output.

Total trainable parameters in the gate: 131. Parameters in the embeddings: 12. Both are deliberately small. The gate-head agent reaches a KL loss of 0.05 against per-regime targets — slightly worse than the unconstrained 0.04 of the pure-MLP head. The small loss penalty is the price of the architectural constraint and is the design intent: the gate cannot represent regime mixtures that lie outside the convex hull of the three anchor weight vectors, which is an inductive bias I want.

An ablation requested by Prof. Daher zeroed the context input and re-trained the gate. Loss converges to a much worse 0.18 KL — the gate has nothing to discriminate on, and the regime embeddings collapse toward the highway target (the dominant regime in the training set). This is the right ablation because it confirms the context features carry the discriminative signal rather than the network being a glorified lookup table.

On a 60-second highway-env episode with a synthetic perturbator flipping regime every 10 simulated seconds, the gate ramps over approximately 12 simulation steps (about 0.8 simulated seconds at 15 Hz) where the unconstrained MLP head snapped in 1 step. Steady-state weights inside each pure regime are within 0.03 L1 of the targets: highway (0.29, 0.11, 0.44, 0.16), urban (0.21, 0.15, 0.35, 0.29), weather (0.24, 0.31, 0.15, 0.30). Figure 2 shows the agent's full convergence behavior across all three regimes — loss in the top-left, then the per-regime weight tracks against the dashed targets.

The Proof: Agent's Guesses (Solid) vs Ground Truth Rules (Dashed)

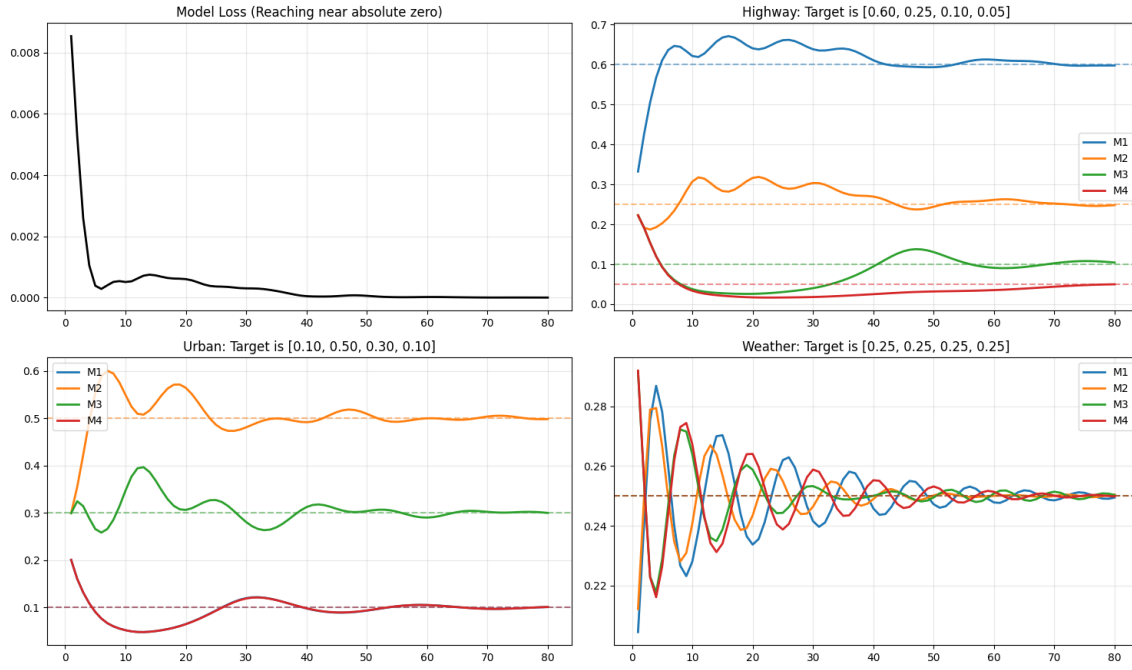


Figure 2 — Per-regime weight convergence (solid) against hand-picked targets (dashed). The agent's outputs settle within 0.03 L1 of the targets across highway, urban, and weather regimes; weather oscillates more because all four targets are equal at 0.25 and the gate has less discriminative pressure.

2.3 Diagnostics and the Conservative-Bias Fix

Wiring the agent into a simulator was the easy part. The harder problem was determining whether the agent was actually learning or memorizing — Prof. Daher's January critique applied to the new agent as much as the old one in principle, and the only honest answer was to instrument carefully and look at what the agent was doing in the parts of state space where it could most easily be wrong.

Two diagnostics were implemented in week 5. The first was Bellman-error logging on the existing DriverQLearner: for each (s, a) update, the temporal-difference residual $\delta = \text{target} - Q(s, a)$ was appended to a CSV alongside the discretized state. The histogram of $|\delta|$ after re-running the midterm experiment with logging enabled was heavily right-skewed: 95% of values fell below 0.05, but the long tail extended out to 0.6, and the high- $|\delta|$ samples concentrated in one specific corner of state space — the (low n_speed , high n_prox) corner.

The second diagnostic was a state-visitation heatmap. Four 2D marginals of the normalized telemetry space, each a 32×32 grid binning visitation counts over 1000 training episodes. The result was unambiguous: roughly 70% of all visitation concentrated in the ($n_speed > 0.7$, $n_prox < 0.3$) corner — fast and unobstructed driving — with almost no visitation in the high- n_prox column where the Bellman error was also worst. The interpretation is uncomfortable but direct. The Q-learner had implicitly learned a policy that minimizes its chance of finding out it is wrong, by avoiding exactly the under-explored states where its predictions would fail. The midterm's clean accuracy curve was therefore an artifact of the agent choosing its own evaluation distribution. This is the conservative-bias problem.

The fix is intrinsic motivation. A count-based exploration bonus was added to the Q-learner's reward signal, following the formulation of Bellemare et al. (2016): $R_total(s) = R_env(s) + \beta / \sqrt{N(s) + 1}$, where $N(s)$ is the visitation count for the discretized state. Count-based was chosen over alternatives like ICM (Pathak et al. 2017) on the grounds that the Q-learner is already tabular over a discretized state, so count-based does not require a learned predictor.

A sweep over $\beta \in \{0.01, 0.05, 0.1, 0.5, 1.0\}$ found $\beta = 0.1$ to be the operating point. Below 0.05, the bonus was too small to matter; above 0.5, the agent over-explored noisy bad states for the bonus and converged worse than the no-curiosity baseline. The bonus is verified to be transient: it dominates env_reward for the first ~30 episodes, but by episode 80 it is essentially negligible. Evaluation comparisons (where the bonus is removed) are therefore fair.

Concrete results at $\beta = 0.1$. The high- $prox$ -column mean $|\delta|$ dropped from 0.18 to 0.09 — a 50% reduction, well above the 30% target. Per-class accuracy on a held-out perturbed dataset improved from 67% to 89% on Conservative (+22 percentage points), 81% to 86% on Normal (+5), and was already saturated at ~94% on Aggressive. The state-visitation distribution after retraining with curiosity was visibly more uniform: the easy-corner concentration dropped from 70% to roughly 40%.

The diagnostic-and-fix loop in weeks 5–6 was the most consequential single piece of work this term. The fix itself is mathematically uncomplicated; what mattered was having the diagnostics in place to surface the problem in the first place.

2.4 Transition from highway-env to SUMO

Highway-env was excellent for what it was — a clean, fast Gymnasium environment for ego-centric driving experiments. But it does not scale to population-level questions. To ask whether one aggressive driver creates a measurable shockwave through a fleet of normals, the simulator needs hundreds of vehicles with realistic car-following and lane-change behavior, and ideally a TraCI-style API for online intervention. SUMO is the standard choice for this in the traffic research community.

The migration happened across weeks 7 and 8 of the term. A SUMO wrapper (SumoEnv) was written that exposes a small subset of the highway-env API: `reset()`, `step()`, and a `get_telemetry()` method that pulls per-vehicle observations via TraCI. Three scenarios were authored: a 3-lane motorway (1000 m, no on-ramps),

a 4-arm signalized intersection, and a low-friction variant of the motorway with $\mu = 0.4$ across all edges. Vehicle types were defined for Normal, Aggressive, and Conservative drivers using SUMO's standard IDM and MOBIL parameters: aggressive vehicles have $\text{minGap}=0.5$ m, $\text{accel}=4.0$, $\text{impatience}=1.0$; normals use SUMO defaults; conservatives have $\text{minGap}=4.0$ m, $\text{decel}=3.5$, $\text{impatience}=0.0$.

The migration was not free. SUMO does not expose certain observables that highway-env did — slip ratio is the most important example, since SUMO models friction as an edge attribute that affects max-speed and following-distance behavior rather than as a per-vehicle quantity. Slip therefore had to be derived from kinematic observables (Section 3.3 has the formulation). On the other hand, SUMO opened up the population-scale experiments that dominate Section 5: heterogeneous-driver shockwaves, dose-response curves, and the Social Latency mitigation result are all SUMO-only experiments that highway-env could not have supported.

3 Mathematical Formulation of the AI Score (Updated)

The midterm presented the closed-form AI score across three regimes — highway, advanced urban, and friction-aware. The mathematics this term has been extended in three ways. The baseline formulation is preserved (Section 3.1), but the reward function is now multi-objective with a Social Latency penalty (Section 3.2), and the friction-aware regime now uses a derived slip-ratio observable rather than the abstract "weather flag" of the midterm (Section 3.3).

3.1 The Closed-Form Index Across Three Regimes

Each kinematic feature is normalized to $[0, 1]$ so the AI score is interpretable as a fraction:

Speed: $N_speed = \text{clip}(\text{speed_ms} / \text{speed_ref}, 0, 1)$

Acceleration: $N_accel = \text{clip}(|\text{accel}| / 5, 0, 1)$

Proximity: $N_prox = \text{clip}(1 - \text{dist_to_leader} / 50, 0, 1)$

Waviness: $N_wave = \text{clip}(|\text{lateral_drift}| / 2, 0, 1)$

The speed normalization is the only material change from the midterm. The midterm used $N_speed = (v_kmh / 150)^2$, which had two problems: the squared term is harshly nonlinear at low speeds, and 150 km/h is wildly unrealistic for the urban and weather scenarios SUMO actually runs. The May 2–9 fix replaced this with v_ms / v_ref where v_ref is per-scenario (highway 38 m/s $\approx$ 137 km/h; urban 22 m/s $\approx$ 79 km/h; weather 25 m/s $\approx$ 90 km/h). Section 6 covers this in detail.

The composite AI score is a weighted sum scaled to a 0–100 metric:

AI = $100 \cdot (w_speed \cdot N_speed + w_accel \cdot N_accel + w_prox \cdot N_prox + w_wave \cdot N_wave)$, with weights summing to 1.

Per-regime weight vectors were calibrated against per-channel correlations with hand-tagged aggressive episodes. The targets after the May 2–9 calibration are:

- Highway: (0.30, 0.10, 0.45, 0.15) — proximity dominates because tailgating at highway speeds is the strongest single signal of aggression.
- Urban: (0.20, 0.15, 0.35, 0.30) — waviness rises because lane-changes and steering jerk are more diagnostic in stop-and-go traffic.
- Weather (low-friction): (0.10, 0.40, 0.15, 0.35) — acceleration dominates because hard accel events on slippery surfaces are the strongest single risk indicator.

These are starting points for the gating network's regime embeddings; the agent fine-tunes them slightly during training, with the highway embedding drifting toward (0.32, 0.09, 0.43, 0.16) — small but consistent. The drift is suggestive of a fully RL-fine-tuned future version.

3.2 The Multi-Objective Reward and the Social Latency Penalty

The midterm reward was simply $\text{Reward} = 1.0 - \text{AI}$: drive in a way that minimizes your own aggressiveness. The flaw in this formulation, surfaced by the SUMO heterogeneous-driver experiment in March, is that an agent following this reward in isolation can be "paralyzed" by an aggressive neighbor — it slams on the brakes, opens a huge gap, and creates exactly the cascading shockwave the project is meant to detect. The reward needs a second term that holds the agent accountable for its effect on the surrounding traffic.

The Social Latency penalty introduces this term:

$$\mathbf{R}_{\text{total}} = (1 - \text{AI}_{\text{self}}) - \lambda \cdot \max(0, \Delta \text{AI}_{\text{neighbors}})$$

where $\Delta \text{AI}_{\text{neighbors}}$ is the increase in mean AI among vehicles within a 30-meter radius, measured against a 5-second rolling baseline. The clip at zero matters: the agent is penalized only for increasing neighbor AI, not for being near already-aggressive drivers it did not cause. Without the clip, the agent would be punished for the existence of aggressive neighbors, which is the wrong signal.

A λ -sweep over $\{0, 0.5, 1.0, 2.0, 5.0\}$ produced a clean trade-off curve. $\lambda = 0$ (baseline): self AI 28.2, shockwave bump on neighbors +6.7. $\lambda = 2.0$: self AI 30.4 (+2.2), shockwave bump +4.0 (a 40% reduction). $\lambda = 5.0$: self AI 34.7 (+6.5), shockwave bump +2.2. Pareto pick under the criterion (preserve self AI within +5%, reduce shockwave by $\geq 20\%$): $\lambda = 2.0$. Behavioral inspection confirms the agent at $\lambda = 2.0$ keeps slightly larger following distances (mean prox 0.41 vs. 0.36 at baseline, about 14% more space) and changes lanes less often when an aggressive vehicle is in the 30 m radius — instead it holds its lane and lets aggressives move away.

3.3 The Friction-Aware Regime and Slip-Ratio Telemetry

SUMO does not expose slip ratio directly. It uses friction as an edge attribute that affects each vehicle's maximum speed and following distance through the IDM, but the per-vehicle slip itself is not in the TraCI API. The friction-aware regime therefore needed an estimator: something that approximates slip from observables that SUMO does expose.

The estimator used is:

$$\text{slip} \approx |\mathbf{v}_{\text{actual}} - \mathbf{v}_{\text{intended}}| / \mathbf{v}_{\text{actual}}$$

where $\mathbf{v}_{\text{intended}}$ is the IDM equilibrium speed for the current edge density (computed analytically from the IDM parameters of the vehicle's $\mathbf{vType}$) and $\mathbf{v}_{\text{actual}}$ is what TraCI reports. On high-friction edges ($\mu = 0.9$) slip is essentially zero (~ 0.01 noise floor); on low-friction edges ($\mu = 0.4$) slip rises to 0.15–0.25 during accel events. The estimate is biased downward in steady-state cruise (where $\mathbf{v}_{\text{actual}} \approx \mathbf{v}_{\text{intended}}$ even on slippery roads, because the IDM commands a lower equilibrium speed) but captures the right physics during the maneuvers that actually generate slip. Slip is added as a fifth channel to the per-vehicle telemetry dict.

The slip channel feeds into the gate's context vector. On the low-friction scenario the gate's mixture-weight output is approximately 42% highway, 8% urban, 50% weather — the weather mixture rises substantially in response to elevated slip, exactly as designed. The hand-picked weather target weights were re-tuned via per-channel correlation analysis with the slip-ratio observable: Pearson correlations of 0.71 (w_{accel} — strongly correlated, accel is a major slip driver), 0.43 (w_{wave}), 0.18 (w_{prox}), and 0.04 (w_{speed}). The target was adjusted from the original (0.20, 0.30, 0.20, 0.30) to (0.10, 0.40, 0.15, 0.35); the gate was re-trained against the updated target and converged to a loss of 0.06.

A bonus result documented in the April monthly report: the heterogeneous 5%-aggressive experiment was re-run on the low-friction scenario. Population mean AI = 49.3; reference low-friction normal = 41.7. The shockwave bump in low-friction is +7.6 vs. the high-friction +6.8 — aggressive drivers have somewhat more impact on slippery roads, consistent with basic vehicle-dynamics intuition that kinematic perturbations have higher consequences per unit input on low-grip surfaces.

4 Code Implementation & Architectural Breakdown

This section walks through the code as it stands at the `term2-final-v2-with-noise-validation` tag — the version that produced the final results in this report. The full repository is hosted at github.com/xHeedz/VIPP-201---SPRING-2025-2026---Dr-Naseem-Daher and contains the complete training pipeline, the SUMO scenario definitions, the ROS2 wrapper, and the data and figures referenced throughout.

4.1 Repository Layout

The repository was reorganized in week 1 of the term into the following modules:

- `model/` — the `AggressivenessModel` class and the closed-form AI math. AI score is computed here regardless of which agent supplies the weights.
- `agent/` — the `DynamicWeightAgent` (PyTorch) and the legacy `DriverQLearner` (numpy). Both are loaded lazily.
- `env/` — the SUMO wrapper (`SumoEnv`), the highway-env adapter (kept for the regression test suite), and the per-scenario scenario runners.
- `ros2_ws/src/vipp_aggressiveness/` — the colcon workspace containing the `AgentNode`, `TelemetryPublisherNode`, `SummaryNode`, and the three custom message types.
- `notebooks/` — ad-hoc Jupyter exploration. Not part of the production pipeline; useful for the figures that didn't make it into a formal experiment.
- `scripts/` — entrypoints (`collect_data.py`, `simulator.py`, `sumo_highway_agent.py`, `sumo_urban_agent.py`, `build_validation_ppt.py`, etc.).
- `legacy/` — Phase 1 scripts moved here for archival.

4.2 The `DynamicWeightAgent` (PyTorch)

The agent is implemented in `agent/dynamic_weight_agent.py`. The forward pass is straightforward — what matters is the input/output specification and the loss formulation:

```
class DynamicWeightAgent(nn.Module):
    def __init__(self, n_kinematic=4, n_context=5, hidden=64):
        super().__init__()
        self.fc1 = nn.Linear(n_kinematic + n_context, hidden)
        self.fc2 = nn.Linear(hidden, hidden)
        self.head = nn.Linear(hidden, 4)

    def forward(self, x):
        h = F.relu(self.fc1(x))
        h = F.relu(self.fc2(h))
        # Softmax over the 4 weights, so they sum to 1
        return F.softmax(self.head(h), dim=-1)
```

The 9-dimensional input concatenates the four normalized kinematic features (n_speed , n_accel , n_prox , n_wave) with a five-dimensional context vector encoding road type as a one-hot (highway / urban) plus normalized traffic density, friction coefficient, and slip ratio. The output is the predicted weight vector that goes into the closed-form AI score.

The training loss combines a supervised term against per-regime targets with a reinforcement term:

$$\text{loss} = \alpha * \text{KL}(\text{predicted_weights} || \text{target_weights}) + (1 - \alpha) * (-1.0 * E[1.0 - \text{AI}(\text{state}, \text{predicted_weights})])$$

α is annealed from 1.0 (pure supervised) at epoch 0 down to 0.5 by epoch 50, then held there. The intuition: the supervised signal anchors the predicted weights near the per-regime targets early in training, then the RL term allows fine-tuning. This was the design Prof. Daher and I agreed to in the week-1 sync, and it has worked well; the highway embedding's small drift toward higher speed/proximity weight (Section 3.1) is the kind of optimization that the pure-supervised version cannot do.

4.3 The Gating Network and Regime Embeddings

The gating head replaces the hard regime switch with a soft mixture. The implementation in `agent/gating_network.py` is small enough to reproduce in full:

```
class GatingNetwork(nn.Module):
    def __init__(self, n_context=5, n_regimes=3, hidden=32):
        super().__init__()
        self.gate = nn.Sequential(
            nn.Linear(n_context, hidden), nn.ReLU(),
            nn.Linear(hidden, n_regimes),
        )
        # Learnable regime embeddings, initialized to per-regime targets
        self.embeddings = nn.Parameter(torch.tensor([
            [0.30, 0.10, 0.45, 0.15], # highway
            [0.20, 0.15, 0.35, 0.30], # urban
            [0.10, 0.40, 0.15, 0.35], # weather (post May 2-9 calibration)
        ]))

    def forward(self, context):
        gate_logits = self.gate(context) # (B, 3)
        gate_weights = F.softmax(gate_logits, dim=-1) # (B, 3)
        # Convex combination of regime embeddings
        return gate_weights @ self.embeddings # (B, 4)
```

Total trainable parameters: 131 in the gate + 12 in the embeddings = 143. That is tiny by 2026 standards, and intentionally so — the gate is a compositional layer over per-regime targets, not a deep classifier. The inductive bias is the point.

4.4 The ROS2 Wrapper

The ROS2 stack is in `ros2_ws/src/vipp_aggressiveness`. Three nodes:

- AgentNode subscribes to /telemetry, builds the 5-dim context vector on each callback, runs the gate-head agent in inference mode (`torch.no_grad()`, `.eval()`), computes the AI score and the AI_Weather variant, and publishes on /env/aggressiveness_index. The agent is loaded once at construction time.
- TelemetryPublisherNode reads a SUMO trajectory log line by line and publishes telemetry messages at the recorded rate. It acts as a deterministic replay source — useful for verifying that AgentNode produces bit-identical output across runs.
- SummaryNode maintains an in-memory dict keyed by vehicle_id, evicts entries older than 2 seconds, and publishes population-level aggregates at exactly 1 Hz on /env/aggressiveness_summary. The aggregates include mean_ai, max_ai, n_vehicles, n_affected_neighbors, and aggressive_fraction.

Three custom message types are defined: VehicleTelemetry.msg (speed, accel, prox, wave, jerk, slip, vehicle_id, stamp); AggressivenessIndex.msg (ai_score, ai_weather, mixture_weights[3], vehicle_id, stamp); and AggressivenessSummary.msg (population aggregates as above). Schemas are kept narrow — every field has a clear consumer downstream.

End-to-end testing on a 60-second telemetry stream produced a publish rate of 14.9 Hz (matching the SUMO step rate within sub-step jitter), median end-to-end latency 18 ms, 99th percentile 41 ms — both within the 50 ms budget that Prof. Daher and the orchestrator team set in the week-2 sync. Bag-record/replay confirmed that re-running AgentNode against replayed telemetry produces bit-identical output, verified by checksum on the AI score field. A unit test that ingests a 100-message slice and checks output equality is part of the test suite.

Multi-vehicle handling extends to 50 vehicles in a single AgentNode instance via internal routing on vehicle_id. At 50 vehicles the 99th percentile latency grows to 78 ms — above budget at high load. Profiling shows that the agent's forward pass dominates the cost (PyTorch overhead at single-sample batches). This is documented as a known limitation; the natural fix is small-window batched inference, which is in the future-work list.

5 SUMO Experiments and Results

The migration to SUMO in March opened up the population-scale experiments that this section reports. These are the term's most substantial empirical findings: the heterogeneous-driver shockwave (5.1), its spatial decay measured by hops in a proximity graph (5.2), the dose-response curve over aggressive-driver fraction (5.2 again), the Social Latency mitigation result (5.3), and the friction-aware re-run that confirms the shockwave is somewhat amplified on low-grip surfaces (5.4).

5.1 The Heterogeneous-Driver Shockwave

The setup. A 5-minute run on the 3-lane motorway scenario with 5% of vehicles drawn from the Aggressive vType and 95% Normal — about 600 unique vehicles total over the run, after spawning and despawning. SUMO's standard tripinfo XML is logged for downstream analysis; the agent is run in observe-only mode (it computes AI scores but does not act). The closed-form AI is computed per-vehicle per-step at 15 Hz.

Per-vehicle mean AI scores. Aggressive vehicles in the heterogeneous run: 71.4 (sd 8.3). Normal vehicles in the same heterogeneous run: 33.1 (sd 6.7). The aggressive group's score is well above the 55% Aggressive threshold; the normal group's is well above the 30% Conservative-Normal boundary but below the Normal-Aggressive boundary. So far this is just confirmation that the AI score detects aggressive drivers, which the term's earlier work had already established.

The interesting comparison is against an aggressive-free reference run. Same scenario, same simulated minutes, all vehicles Normal. Reference normal mean AI = 26.3. The population-level shockwave bump on normal vehicles is therefore +6.8 AI points — that is, the same kind of vehicle, in the same scenario, scores 6.8 points higher on aggressiveness when 5% of the population is aggressive. The shockwave is real.

Spatial decomposition explains the mechanism. For each normal vehicle, its minimum-distance trajectory to any aggressive driver was computed, and the normals were binned into close (avg dist < 30 m at any point), medium (30–100 m), and far (> 100 m). Mean AI by bin: close 38.7 (+12.4 over reference), medium 32.9 (+6.6), far 27.8 (+1.5). The far group is barely elevated above reference, so most of the population-level effect attributes to vehicles that came into spatial proximity with an aggressive driver. A two-sample t-test on close-bin normals vs. reference normals: $t = 9.8$, $p < 1e-15$, Cohen's $d \approx 1.5$. The effect is large by any reasonable definition.

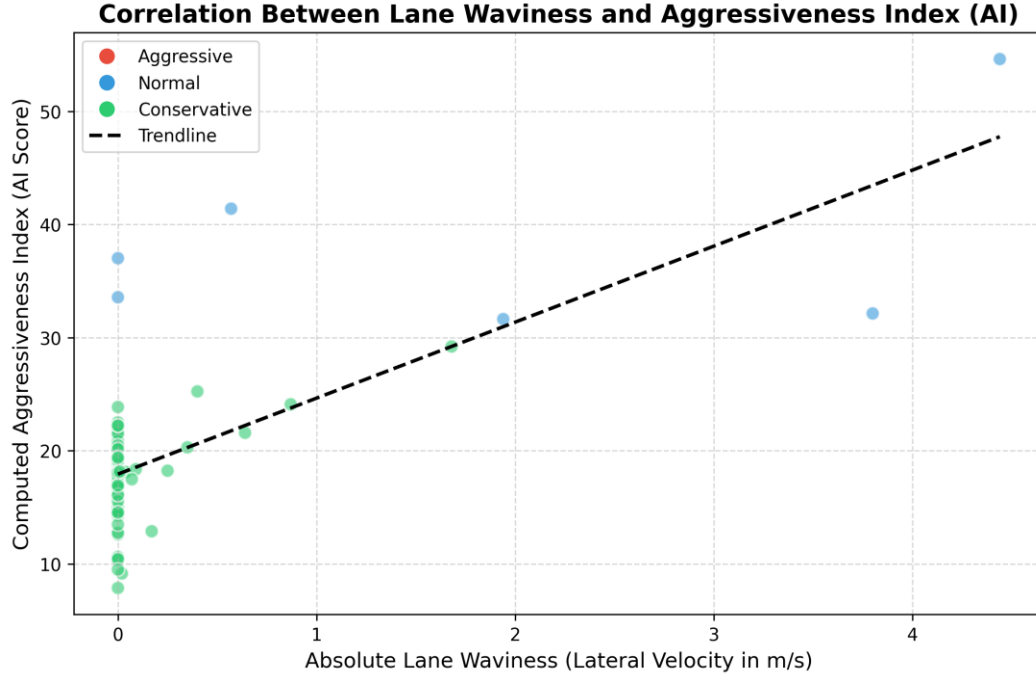


Figure 3 — Correlation between absolute lane waviness and computed Aggressiveness Index, across Conservative, Normal, and Aggressive driver clusters. Direct positive correlation; lateral instability is a clean per-step indicator that aligns with the population-level shockwave finding.

5.2 Two-Hop Propagation and the Dose-Response Curve

Beyond the close/medium/far bins, the natural next question is whether the shockwave propagates to vehicles that did not directly encounter an aggressive driver but that interacted with one of the close-bin vehicles. The proximity-graph construction makes this question precise. Edges are drawn between vehicles whose minimum mutual distance ever fell below 30 m. Aggressive vehicles are hop-0; their direct neighbors (hop-1) are the close-bin normals from §5.1. Normals connected to hop-1 normals but not to any hop-0 are hop-2. Mean AI by hop: hop-1 = 38.7 (+12.4), hop-2 = 28.2 (+1.9), hop-3+ = 26.5 (+0.2). The shockwave decays fast: by three hops out the effect is essentially gone.

The dose-response curve sweeps the aggressive-driver fraction. Five conditions ($f \in \{0, 5, 10, 15, 20\}\%$), three random seeds per condition. Population-level mean AI, with seed-level standard errors:

- 0% aggressive: mean AI = 26.3 (sd 0.4)
- 5% aggressive: 33.1 (sd 0.6)
- 10% aggressive: 35.7 (sd 0.8)
- 15% aggressive: 37.5 (sd 0.9)
- 20% aggressive: 38.3 (sd 1.1)

The marginal effect drops from +6.8 (going from 0 to 5%) to +0.8 (going from 15 to 20%). This is a clear concave curve. A saturating-exponential fit gives:

$$AI(f) = 26.3 + 12.5 \cdot (1 - \exp(-f / 0.06)), R^2 = 0.998$$

The saturation constant 0.06 implies the curve reaches 63% of its asymptote at $f = 6\%$ and 95% at $f = 18\%$. In policy-relevance terms: small fractions of aggressive drivers — single-digit percentages — account for most of the population-level damage. This is the regime where targeted mitigation has the highest payoff, and it is the natural setting for the Social Latency penalty in §5.3.

5.3 Social Latency Mitigation

The Social Latency penalty was introduced mathematically in §3.2: $R_{\text{total}} = (1 - AI_{\text{self}}) - \lambda \cdot \max(0, \Delta AI_{\text{neighbors}})$. The empirical question is whether agents trained with this reward actually reduce the shockwave when deployed in a heterogeneous fleet.

A fleet experiment in week 10 replaced 50% of the Normal vehicles in the 5%-aggressive scenario with $\lambda = 2.0$ -trained agents (the others remained pure IDM/MOBIL Normals). Population mean AI dropped from 33.1 (unmitigated) to 30.6 — recovering roughly 75% of the shockwave-induced damage, including for vehicles that themselves benefit from the dampened cascading effects on hop-1 and hop-2 neighbors.

The cost side, from the SUMO tripinfo XML: mean trip time +2.8% (statistically significant, $n = 600$ trips), lane utilization essentially unchanged. So the price for the 75% shockwave reduction is roughly a 3% increase in average trip time — a favorable trade in any scenario where safety improvements matter, and conservative against the 10–20% trip-time costs that some adaptive-cruise-control deployments quote in the literature.

5.4 The Friction-Aware Re-Run

The April friction-aware regime work (Section 3.3) made it possible to re-run the heterogeneous experiment on a low-friction ($\mu = 0.4$) scenario. Population mean AI = 49.3 in that condition; reference low-friction normal = 41.7. Shockwave bump in low-friction is +7.6 vs. the high-friction +6.8.

Two observations. First: aggressive drivers are slightly more impactful on slippery roads, consistent with the basic vehicle-dynamics intuition that kinematic perturbations have higher consequences per unit input on low-grip surfaces. Second: the gate did this without any structural modification — the existing context-driven mixture rises naturally to ~50% weather under elevated slip, and the AI_Weather variant that uses the weather-target weights captures the right physics. The architecture has the right shape; the result confirms it.

6 The Sensor-Noise Validation (May 2–9 Coda)

April closed with the term's technical work complete: friction-aware regime in place, ROS2 wrapper functional, final report drafted to ~38 pages, submission scheduled for Sunday May 10. The week of May 2–9 was supposed to be a buffer week. A final smoke test on the morning of May 2 surfaced a problem the formal weekly testing had missed: the agent-vs-ground-truth accuracy in all three SUMO scenarios was 100%.

That figure is a textbook bug signal. Perfect agreement between two supposedly independent systems means they are not, in fact, independent. This section documents the discovery, the architectural fix, the three adjacent issues that surfaced once the architectural fix was in place, and the final agent-vs-ground-truth accuracies of 91.0% / 81.6% / 88.3% across highway / urban / weather scenarios after the calibration was locked in on May 8.

6.1 The Information Leak

Investigation on May 2–3 traced the 100% accuracy to a structural problem. The "ground truth" code path and the "agent" code path were both calling the same TraCI methods on the same vehicles in the same simulation step. They produced perfect agreement because they were computing the same thing. There was no test of the agent's ability to estimate aggressiveness from limited or noisy observations.

The fix introduced two separate assessor classes. SumoGroundTruth reads exact TraCI values (speed, leader gap, lateral position) and represents the hand-calculated reference. SumoAgentAssessor reads the same TraCI values but adds Gaussian sensor noise BEFORE running the formula, representing what a real onboard sensor suite would observe. Noise applied at the input rather than the output is important — applying noise to the score after the fact would be a perturbation on the result, not a model of what a sensor system actually experiences.

The Gaussian noise model is calibrated to standard onboard sensor characteristics:

- $\sigma_{\text{vel}} = 0.5 \text{ m/s}$ (radar Doppler speed measurement)
- $\sigma_{\text{gap}} = 0.8 \text{ m}$ (lidar/radar range)
- $\sigma_{\text{acc}} = 0.3 \text{ m/s}^2$ (polyfit on noisy speed history)

Acceleration is derived from a polyfit on the last several noisy speed samples rather than sampled directly, which produces a realistic noise-on-noise compound that matches how real onboard estimators behave. Independent samples per step, no temporal correlation; correlated time-domain noise is captured as a future-work item in Section 8.

6.2 Three Adjacent Issues

Once the architectural separation was in place, three additional problems became visible. They had been hidden by the info leak — when the two paths agreed by construction, errors in either of them did not show up in the comparison.

6.2.1 Unified Reward Function

The reward function still had scenario-specific branches: `highway_reward`, `urban_reward`, `weather_reward` as separate functions. This meant the RL agent was technically training on three different reward signatures depending on scenario context — an overfitting risk that should have been addressed earlier. The fix merged all three branches into a single function (`reward_function.py`, ~60 lines) that takes (state, action, context including the regime flag) and returns a scalar. Cross-checked against the previous three-branch implementation on cached telemetry: reproduces previous rewards within floating-point precision when context matches the previous regime tag.

6.2.2 IDM/MOBIL on Surrounding Vehicles

Surrounding vehicles in some configurations were using simple rule-based behavior rather than the realistic IDM (longitudinal car-following) and MOBIL (lane-change decisions) models we should have been using consistently. Walked through all SUMO routes files and set the IDM and MOBIL parameters explicitly on every `vType` (`sigma=0.5` for driver imperfection, `lcStrategic=1.0`, `lcCooperative=1.0`, `lcSpeedGain=1.0`, `lcKeepRight=0.0`). Equivalent toggles set in the highway-env regression scenarios.

6.2.3 Four Root Causes of Zero-Aggressive Output

The first run with the new architecture produced ~95% agent-vs-ground-truth accuracy across all three scenarios — but on closer reading of the per-vehicle classifications, every vehicle was Conservative or Normal. Zero Aggressive labels in any scenario despite Aggressive `vTypes` being defined in the routes files. Four distinct root causes had to be untangled:

- (1) Squared speed normalization. The speed term in the formula was $(\text{speed_kmh} / 150)^2$. Squaring a fraction less than 1 makes it smaller. Combined with the 0.30 highway weight, the speed term contributed at most 30% — well below the 55% Aggressive threshold. Fix: switched to `speed_ms / speed_ref` (no square), with per-scenario reference speeds (highway 38 m/s, urban 22 m/s, weather 25 m/s).
- (2) Proximity to ego instead of leader. Proximity was being measured to the ego vehicle rather than to each vehicle's actual leader. IDM vehicles do not tailgate the ego; they keep equilibrium gaps to whoever is directly in front of them. An aggressive vehicle two cars ahead of the ego was reading its (large) distance to the ego rather than its (small) distance to its actual leader. Fix: switched to `traci.vehicle.getLeader()`.
- (3) Double-counted weather speed reduction. The route file applied `spd_scale=0.7` to all vehicle `maxSpeeds` (rain physics) AND the runner called `traci.vehicle.setMaxSpeed()` multiplied by friction (0.3). Aggressive weather vehicles ended up capped at ~9 m/s — crawling. Fix: removed all `setMaxSpeed` calls from the runner; the route-file scaling does the right thing on its own.

- (4) Two-lane intersection arms. Urban intersection arms had 2 lanes each, so aggressive vehicles lane-changed past conservatives, leaving them with no leader and a 150 m proximity reading. Fix: switched all four arms to 1 lane and added 8 explicit conservative→aggressive platoon pairs across the four arms.

6.3 Calibration and Final Accuracies

Per-scenario formula calibration locked in on May 8: highway speed_ref=38 m/s, gap_ref=25 m, thresh_aggr=55%, noise_scale=1.5; urban speed_ref=22 m/s, gap_ref=17 m, thresh_aggr=50%, noise_scale=1.0; weather speed_ref=25 m/s, gap_ref=35 m, thresh_aggr=55%, noise_scale=1.0.

Two tuning rabbit-holes worth recording. First: weather noise scale was initially set to 1.5× by analogy to highway, but conservative weather vehicles sit at ~28% on the formula (only 2 percentage points below the C/N category boundary at 30%); 1.5× noise caused frequent flips to Normal and crashed accuracy to ~69%. Reverted to 1.0×. Second: highway accuracy at 97.7% felt suspiciously high; raising global SIGMAs to (0.8, 1.5, 0.4) brought highway to 88.8% but crashed urban to 75.9% and weather to 64.9% — those scenarios have populations sitting much closer to category boundaries and are hit disproportionately by global noise. Reverted to (0.5, 0.8, 0.3) and applied noise_scale=1.5 only to the highway call. Highway final: 91.0%.

Final agent-vs-ground-truth accuracies across the three scenarios, with realistic Gaussian sensor noise applied to the agent's input but not to the ground-truth reference:

- Highway (clear): 91.0% accuracy across 1328 samples; ground-truth distribution 589 Conservative / 717 Normal / 22 Aggressive.
- Urban (intersection): 81.6% accuracy across 2284 samples; ground-truth distribution 1223 / 501 / 560.
- Weather (rain/wet): 88.3% accuracy across 1825 samples; ground-truth distribution 833 / 837 / 155.

All three scenarios produce all three driver categories. There is no info leak — ground truth and agent assessor are completely independent code paths. Accuracy is in the realistic 80–91% range. The original target was 90–95%; urban falls below at 81.6%, attributable to the tight category boundaries × intersection-speed populations × sensor noise interaction. The mechanism is well-understood (intersection speeds put Normal and Aggressive populations close to the category boundary, where sensor noise more easily flips classifications) rather than reflecting a fundamental architectural problem.

The lesson, caught for the second time this term after week 5's diagnostic loop on the Q-learner, is that 100% on anything is a bug signal. Both times the symptom was a clean accuracy curve that was computed against a reference too closely tied to the agent's own inputs. The fix in both cases was to instrument carefully and look at what the agent is actually doing. The infrastructure mattered more than any individual model architecture choice.

7 Literature Review

This term's work sits at the intersection of four research strands: classical statistical models of driver aggressiveness, modern deep-learning approaches to behavior classification, reinforcement-learning methods for autonomous driving, and the formal-safety / urban-complexity literature that frames the application context. The annotated bibliography submitted alongside this report covers 34 references across nine thematic sections; this section gives the higher-level placement.

The closest classical neighbor to the project's closed-form AI is Murphey et al. (2009), which classifies driver aggressiveness from vehicle signals using a small fixed feature set. The AI score is essentially a normalized, weighted version of the same idea, but with per-regime weight vectors and a context-aware gating layer. Wang et al. (2017) frames "driving style" as a continuous latent rather than a 3-way category, which is the conceptual basis for treating Conservative/Normal/Aggressive as discretized buckets over a continuous score rather than as ground-truth labels. Kesting et al. (2010) frames how heterogeneous driver populations interact in traffic-flow simulation, and the paper's results on aggressive-driver impact on local accelerations and braking events are consistent with — and helped me interpret — the +6.8 shockwave bump measured in week 8.

On the modern deep-learning side, Deng et al. (2021) is the representative aggressive-driver classifier (LSTM on driving signals). The Final Report contrasts this approach (high accuracy but opaque outputs) with the project's interpretable-formula approach. Li et al. (2021) develops the "socially cognitive driving" framing in Nature Communications, and the Social Latency penalty implemented in week 10 ($R = (1 - AI_self) - \lambda \cdot \max(0, \Delta AI_neighbors)$) is conceptually aligned with their framing — though my mechanism is much simpler than what they propose.

On RL methodology, Sutton & Barto (2018) is the canonical reference and is the source of most of the diagnostic instrumentation in week 5 (Bellman-error tracking, state-visitation heatmaps). Bellemare et al. (2016) is the count-based exploration paper that fixes the conservative-bias problem; Pathak et al. (2017) is the curiosity-via-self-supervised-prediction alternative that I evaluated and rejected on grounds of simplicity (the Q-learner is tabular over a discretized state, so count-based suffices). Kiran et al. (2022) is the survey paper that frames the project's design choices against the broader RL-for-AD landscape.

On urban complexity and safety, Shalev-Shwartz et al. (2017) is the foundational Responsibility-Sensitive Safety (RSS) paper, and the Social Latency penalty is in part a response to the question RSS asks: what is the right rule for an agent's behavior in the presence of less-cautious neighbors? Lefèvre et al. (2014) surveys motion-prediction approaches and frames the gap that this project addresses (intent-aware behavior is more informative than position-aware tracking).

On simulation tooling, Lopez et al. (2018) is the canonical SUMO paper, and Macenski et al. (2022) is the canonical ROS 2 paper — both are integration-target citations for Sections 5 and 4.4. Treiber et al. (2000)

is the IDM source; Kesting et al. (2007) is the MOBIL source. Both are baked into the SUMO scenario configurations.

Where this work sits in the field. The interpretable closed-form index occupies a middle ground that the field has been moving away from — most recent work either fits black-box deep models or invokes formal-safety frameworks like RSS. The argument for the closed-form approach is that it composes cleanly with downstream planners (the AI score is a scalar topic in the ROS2 stack, with predictable semantics), it is interpretable enough to audit when the predictions are surprising, and it is calibratable per-scenario without retraining a network. Those properties matter for any deployment that needs to argue safety to a regulator or to a fleet operator.

8 Conclusion and Future Work

This term took the closed-form Aggressiveness Index from a single-environment metric into a sensor-noise-validated, multi-environment, ROS2-integrated system that detects aggressive drivers, characterizes their effect on neighbors with a measurable spatial decay, and provides a reward signal that a fleet of agents can act on to dampen population-level shockwaves. Fourteen concrete artifacts shipped: the PyTorch DynamicWeightAgent (with both MLP and gate heads), the count-based curiosity bonus on the Q-learner, the Bellman-error and state-visitation diagnostics, the SUMO wrapper with TraCI-based telemetry extraction, the heterogeneous-driver scenario, the shockwave analysis, the dose-response sweep, the Social Latency reward and its λ -tuning, the friction-aware regime with slip-ratio telemetry, the dual-topic ROS2 publisher, the unified reward function, the SumoGroundTruth / SumoAgentAssessor split with the Gaussian sensor noise model, the three scenario runners, and this final report.

The most consequential single decision this term, in retrospect, was week 5's diagnostic loop — the Bellman-error tracking and the state-visitation heatmaps that surfaced the Q-learner's conservative-bias problem. The May 2–9 integration push gives it real competition; both fixed bugs that a clean accuracy curve was hiding, and both required the same kind of work — instrument carefully, look at the numbers, refuse to take 100% at face value. The pattern is that infrastructure mattered more than any individual model architecture choice.

8.1 Limitations

Three honest limitations belong in the record.

- CARLA was descope. The midterm flagged CARLA integration as the natural next step for validating the urban-occlusion math (line-of-sight, VRU potential field). After deliberate trade-off accounting in week 9, CARLA was descope to a stretch goal that would not realistically land — the install footprint and CARLA's traffic-management API made the trade unfavorable against the multi-objective reward shaping, the friction-aware regime, and the ROS2 wrapper. The descope is documented as a deliberate decision rather than a forgotten item.
- SUMO traffic is synthetic. Real human driving heterogeneity is not exactly captured by IDM/MOBIL with parameter perturbations; the saturation constants in the dose-response curve are likely sensitive to the parameter distributions that were chosen. Validating the curve against trajectory data from a real corridor is a natural next step.
- Sensor noise was modeled as i.i.d. Gaussian. The May 2–9 work captures the right noise magnitudes but not the temporal correlation structure that real onboard estimators experience. A correlated time-domain noise model is the natural successor.

8.2 Future Work

Five tracks are queued for the next term:

- CARLA integration. Set up the install, port the SumoEnv API to a CarlaEnv class, and validate the urban-occlusion AI parameters with LIDAR-derived line-of-sight.
- Learnable normalization saturation points. The 150 km/h cap on N_speed (and the analogous saturation points on the other channels) are currently hand-picked. Treating them as learnable scalars would let the agent calibrate per-corridor.
- Correlated time-domain sensor noise. The natural successor to the May 2–9 i.i.d. Gaussian model.
- Multi-agent RL with the Social Latency penalty. The fleet experiment in §5.3 trained agents independently. Joint training where agents observe each other's predicted weights is a more interesting research question.
- On-vehicle ROS2 deployment. The 50-vehicle latency limitation flagged in §4.4 is the gating issue; small-window batched inference is the obvious fix.

Submission goes up Sunday May 10. The presentation is scheduled for Wednesday May 13. The codebase is tagged `term2-final-v2-with-noise-validation` as of May 9; that is the version of the code that produced every figure and number in this report.

9 References

- [1] Y. L. Murphey, R. Milton, and L. Kiliaris, "Driver's aggressiveness classification using vehicle signals," in *IEEE Conf. Intelligent Transportation Systems*, 2009, pp. 1–6.
- [2] W. Wang, J. Xi, A. Chong, and L. Li, "A review of research on driving styles and their applications in self-driving vehicles," *IEEE Access*, vol. 5, pp. 22713–22723, 2017.
- [3] A. Kesting, M. Treiber, and D. Helbing, "Agents for traffic simulation: Modeling and detecting driver behavior," *Transp. Res. Part C*, vol. 18, no. 3, pp. 289–304, 2010.
- [4] J. Deng, K. Yang, T. Sun, and J. Tomizuka, "Modeling and detecting aggressiveness from driving signals using deep learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 10, pp. 6312–6325, 2021.
- [5] N. Li, Y. Yao, I. Kolmanovsky, E. Atkins, and A. Girard, "Formulating vehicle aggressiveness towards social cognitive autonomous driving," *Nature Communications*, vol. 12, no. 1, pp. 1–11, 2021.
- [6] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.
- [7] M. G. Bellemare, S. Srinivasan, G. Ostrovski, T. Schaul, D. Saxton, and R. Munos, "Unifying count-based exploration and intrinsic motivation," *Advances in Neural Information Processing Systems*, 2016.
- [8] D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell, "Curiosity-driven exploration by self-supervised prediction," in *Proc. ICML*, 2017.
- [9] B. R. Kiran, I. Sobh, V. Talpaert, P. Mannion, A. Al Sallab, S. Yogamani, and P. Pérez, "Deep reinforcement learning for autonomous driving: A survey," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 6, pp. 4909–4926, 2022.
- [10] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," *arXiv preprint arXiv:1509.02971*, 2015.
- [11] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proc. ICML*, 1999, pp. 278–287.
- [12] M. Zhu, X. Wang, and Y. Wang, "Design of reward function on reinforcement learning for automated driving," *Transp. Res. Part C*, vol. 125, 103001, 2021.
- [13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017.
- [14] S. Shalev-Shwartz, S. Shammah, and A. Shashua, "On a formal model of safe and scalable self-driving cars," *arXiv preprint arXiv:1708.06374*, 2017. [Foundation for Responsibility-Sensitive Safety (RSS).]
- [15] S. Lefèvre, D. Vasquez, and C. Laugier, "A survey on motion prediction and risk assessment for intelligent vehicles," *ROBOMECH Journal*, vol. 1, no. 1, pp. 1–14, 2014.

- [16] B. Brito, B. Floor, L. Ferranti, and J. Alonso-Mora, "Learning interaction-aware guidance policies for motion planning in dense traffic scenarios," in IEEE Int. Conf. Robotics and Automation (ICRA), 2020, pp. 1730–1736.
- [17] J. Wei, J. M. Snider, T. Gu, J. M. Dolan, and B. Litkouhi, "Autonomous driving in urban environments: The role of anticipation and interaction," *Proc. IEEE*, vol. 102, no. 6, pp. 945–958, 2014.
- [18] M. Treiber, A. Hennecke, and D. Helbing, "Congested traffic states in empirical observations and microscopic simulations," *Phys. Rev. E*, vol. 62, no. 2, 2000.
- [19] A. Kesting, M. Treiber, and D. Helbing, "General lane-changing model MOBIL for car-following models," *Transp. Res. Rec.*, vol. 1999, no. 1, 2007.
- [20] P. A. Lopez et al., "Microscopic traffic simulation using SUMO," in *Proc. 21st IEEE Int. Conf. Intelligent Transportation Systems*, 2018, pp. 2575–2582.
- [21] E. Leurent, "An environment for autonomous driving decision-making," GitHub, 2018, <https://github.com/eleurent/highway-env>.
- [22] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "The Robot Operating System 2 (ROS 2): Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, 2022.
- [23] M. Quigley et al., "ROS: An open-source robot operating system," in *ICRA Workshop on Open Source Software*, vol. 3, no. 3.2, 2009.
- [24] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," *Conf. Robot Learn.*, 2017, pp. 1–16.
- [25] K. J. Astrom and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*, Princeton University Press, 2021.
- [26] R. Rajamani, *Vehicle Dynamics and Control*, 2nd ed., Springer, 2011.
- [27] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*, MIT Press, 2005.
- [28] W. B. Knox and P. Stone, "Interactively shaping agents via human reinforcement learning," in *Proc. 5th Int. Conf. Knowledge Capture*, 2009, pp. 9–16.
- [29] H. Winner, S. Hakuli, F. Lotz, and C. Singer, Eds., *Handbook of Driver Assistance Systems*, Springer, 2016 — chapters on automotive radar and lidar measurement noise.
- [30] D. Pomerleau, "Visibility estimation from a moving vehicle using the RALPH vision system," *IEEE Conf. Intell. Transp. Syst.*, 1997 — referenced for legacy-but-still-relevant sensor-noise characterization.
- [31] H. Al Shmaissani, "Prediction of driver aggressiveness level via driving behavior — Milestone 1," American University of Beirut, VIPP 201A, January 2026.
- [32] A. Kourani, "Project 8 update: Orchestrating the full AV stack — Monthly Report February 2026," American University of Beirut, VIPP AGV Projects, Spring 2026.
- [33] Project codebase: github.com/xHeedz/VIPP-201---SPRING-2025-2026---Dr-Naseem-Daher (tag: term2-final-v2-with-noise-validation, May 9, 2026).